Full AdaBoost Algorithm

Given training data $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$ and number of rounds M:

Step 0: Start with equal attention on all points

Initialize sample weights:

$$w_i^{(1)} = \frac{1}{n}, \quad i = 1, \ldots, n$$

Intuition: at the beginning, every training example is equally important.

Step 1: Train one weak learner on the current weighted data

For round $j = 1, \ldots, M$, fit weak learner $h_j(x) \in \{-1, +1\}$ using weights $w_i^{(j)}$.

Intuition: the learner is encouraged to focus on examples with larger current weights.

Step 2: Measure weighted error

Compute:

$$\epsilon_j = \frac{\sum_{i=1}^n w_i^{(j)} \mathbf{1}\{y_i \neq h_j(x_i)\}}{\sum_{i=1}^n w_i^{(j)}}$$

Intuition: mistakes on high-weight points count more than mistakes on low-weight points.

Step 3: Convert error into learner vote weight

Set:

$$\alpha_j = \frac{1}{2} \log\left(\frac{1 - \epsilon_j}{\epsilon_j}\right)$$

Intuition (more to come below!):

- Small ϵ_j (good learner) $\Rightarrow$ large positive α_j.
- $\epsilon_j = 0.5$ (random) $\Rightarrow \alpha_j = 0$.
- $\epsilon_j > 0.5$ (worse than random) $\Rightarrow$ negative α_j.

Step 4: Update sample weights (upweight mistakes, downweight correct)

Unnormalized update:

$$\tilde{w}_i^{(j+1)} = w_i^{(j)} \exp\big(-\alpha_j y_i h_j(x_i)\big)$$

Since $y_i h_j(x_i) = +1$ when correct and -1 when incorrect:

- Correctly classified: multiply by $e^{-\alpha_j}$ (weight decreases).
- Misclassified: multiply by $e^{+\alpha_j}$ (weight increases).

Normalize to keep a distribution:

$$w_i^{(j+1)} = \frac{\tilde{w}_i^{(j+1)}}{\sum_{k=1}^{n} \tilde{w}_k^{(j+1)}}$$

Intuition: the next learner is pushed toward the hard cases from this round.

Step 5: Build the final ensemble

After M rounds, define score:

$$F(x) = \sum_{j=1}^{M} \alpha_j h_j(x)$$

Final classifier:

$$H(x) = \text{sign}(F(x))$$

Intuition: each weak learner votes; stronger learners (larger α_j) get louder votes.

Compact pseudocode view

1. Initialize $w_i^{(1)} = 1/n$.

2. For $j = 1$ to M:
   - Train h_j with weights $w^{(j)}$.
   - Compute ϵ_j.
   - Compute $\alpha_j = \frac{1}{2} \log \frac{1-\epsilon_j}{\epsilon_j}$.
   - Update $w_i^{(j+1)} \propto w_i^{(j)} e^{-\alpha_j y_i h_j(x_i)}$ and normalize.
3. Output $H(x) = \text{sign}\left(\sum_{j=1}^{M} \alpha_j h_j(x)\right)$.

1. Setup: What AdaBoost Is Doing

AdaBoost builds an additive model:

$$F(x) = \sum_{j=1}^{M} \alpha_j h_j(x)$$

Final prediction:

$$\text{sign}(F(x))$$

where:

- $h_j(x) \in \{-1, +1\}$ is the weak learner.
- $y_i \in \{-1, +1\}$ are true labels.
- α_j is the vote weight of classifier j.

The core loop in each boosting round can be read as:

1. **Fit a weak learner on weighted data.**
   Use the current sample weights so the learner focuses more on points that are currently hard.
2. **Compute the learner's weighted error ϵ_j.**
   Errors on high-weight points count more than errors on low-weight points.
3. **Convert error into vote strength α_j.**
   Better learners get larger positive votes; random learners get near-zero vote.
4. **Reweight the training points for the next round.**
   Increase weights on misclassified points and decrease weights on correctly classified points.

Repeated over rounds, this builds the additive score $F(x)$ and final prediction $\text{sign}(F(x))$.

2. Minimize Exponential Loss

AdaBoost minimizes the exponential loss:

$$L = \sum_i \exp\big(-y_i F(x_i)\big)$$

At round j, we update:

$$F_j(x) = F_{j-1}(x) + \alpha_j h_j(x)$$

We want to choose α_j that minimizes:

$$L(\alpha_j) = \sum_i \exp\Big(-y_i\big(F_{j-1}(x_i) + \alpha_j h_j(x_i)\big)\Big)$$

3. Introduce Instance Weights

Define:

$$w_i = \exp\big(-y_i F_{j-1}(x_i)\big)$$

Now the loss becomes:

$$L(\alpha_j) = \sum_i w_i \exp\big(-\alpha_j y_i h_j(x_i)\big)$$

Since $y_i h_j(x_i) \in \{-1, +1\}$, split into:

- Correct predictions: $y_i = h_j(x_i)$
- Incorrect predictions: $y_i \neq h_j(x_i)$

Define weighted error:

$$\epsilon_j = \frac{\sum_i w_i \, \mathbf{1}\{y_i \neq h_j(x_i)\}}{\sum_i w_i}$$

$$L(\alpha_j) = e^{\alpha_j} \sum_i w_i \, \mathbf{1}\{y_i \neq h_j(x_i)\} + e^{-\alpha_j} \sum_i w_i \, \mathbf{1}\{y_i = h_j(x_i)\}.$$

Using
$\sum_i w_i \, \mathbf{1}\{y_i \neq h_j(x_i)\} = \epsilon_j \sum_i w_i$

and

$\sum_i w_i \, \mathbf{1}\{y_i = h_j(x_i)\} = (1 - \epsilon_j) \sum_i w_i,$
we get:

Then:

$$L(\alpha_j) = \left(\sum_i w_i \right) \left[\epsilon_j e^{\alpha_j} + (1 - \epsilon_j) e^{-\alpha_j} \right]$$

4. Minimize with Respect to α_j

Minimize:

$$\epsilon_j e^{\alpha_j} + (1 - \epsilon_j) e^{-\alpha_j}$$

Take derivative with respect to α_j and set to zero:

$$\epsilon_j e^{\alpha_j} - (1 - \epsilon_j) e^{-\alpha_j} = 0$$

Solve:

$$\epsilon_j e^{\alpha_j} = (1 - \epsilon_j) e^{-\alpha_j}$$

Multiply by e^{α_j}:

$$\epsilon_j e^{2\alpha_j} = 1 - \epsilon_j$$

Thus:

$$e^{2\alpha_j} = \frac{1 - \epsilon_j}{\epsilon_j}$$

Taking logs:

$$2\alpha_j = \log \left(\frac{1 - \epsilon_j}{\epsilon_j} \right)$$

Therefore:

$$\boxed{\alpha_j = \frac{1}{2} \log \left(\frac{1 - \epsilon_j}{\epsilon_j} \right)}$$

5. Intuition Behind the Formula

Case 1: Random Guess

If $\epsilon_j = 0.5$:

$$\alpha_j = 0$$

A random classifier gets zero vote.

Case 2: Better Than Random

If $\epsilon_j < 0.5$:

$$\alpha_j > 0$$

Stronger learners receive more weight.

Case 3: Worse Than Random

If $\epsilon_j > 0.5$:

$$\alpha_j < 0$$

The learner gets negative weight (equivalent to flipping predictions).

Case 4: Nearly Perfect

If $\epsilon_j \to 0$:

$$\alpha_j \to \infty$$

The classifier dominates the ensemble.

6. Deeper Interpretation

The quantity

$$\log\left(\frac{1-\epsilon_j}{\epsilon_j}\right)$$

is the **log-odds of correctness**.

Thus:

$$\alpha_j = \frac{1}{2} \times \text{log-odds}$$

AdaBoost assigns weight proportional to confidence in being better than random.

7. Big Picture View

This formula is not heuristic.

It is:

- The exact minimizer of exponential loss.
- Equivalent to forward stagewise additive modeling.
- Interpretable as gradient descent in function space.
- Closely related to logistic regression.

Summary

AdaBoost defines

$$\alpha_j = \frac{1}{2} \log \left(\frac{1 - \epsilon_j}{\epsilon_j} \right)$$

because this is the value that exactly minimizes the exponential loss at each boosting step.

It naturally:

- Gives zero weight to random learners.
- Rewards strong learners.
- Penalizes weak learners.
- Interprets confidence through log-odds.

This is why the formula is elegant, principled, and theoretically grounded.